Desafio Técnico: Nexo Assist, o agent do service desk de TI

PRD do desafio prático do processo seletivo

Contexto e objetivo

A **Nexo** é uma empresa com cerca de 2.000 funcionários. Quando algo de TI dá errado, ou quando alguém precisa de um acesso ou equipamento, o funcionário abre um chamado no service desk. Hoje isso é feito por formulário, e o resultado é conhecido: chamados abertos para dúvidas que a central de ajuda já responde, chamados duplicados, prioridade errada, informações faltando e analistas gastando boa parte do dia com triagem.

A Nexo quer o **Nexo Assist**: um agent de atendimento num chat. O funcionário descreve o que está acontecendo com as próprias palavras, e o agent consulta os chamados dele, busca na central de ajuda, abre ou atualiza chamados e chama um analista quando precisa.

O sucesso será medido por:

- **Resolução sem fila:** dúvidas e consultas resolvidas no próprio chat.
- **Chamados bem abertos:** completos, com tipo, categoria e prioridade corretos, e sem duplicidade.
- **Encaminhamento certo:** o que precisa de uma pessoa chega ao analista na hora certa e com contexto suficiente.
- **Transparência:** é possível entender o que o agent fez e por quê em cada resposta.

Usuários e histórias

O usuário principal é o **funcionário da Nexo**, que entra com o e-mail corporativo. Como funcionário, eu quero:

1. Tirar uma dúvida sem precisar abrir chamado.
2. Saber como está um chamado meu, mesmo sem lembrar o número, e saber se ele está atrasado.
3. Complementar um chamado aberto com novas informações.
4. Abrir um chamado só conversando, conferindo os dados antes de ele ser criado.
5. Falar com um analista quando eu preferir ou quando o problema for grave.
6. Voltar outro dia e continuar de onde parei.

Do outro lado está o **analista do service desk** (Suporte N1 e N2). Ele recebe os chamados criados pelo agent e as conversas transferidas. Como analista, eu quero:

1. Receber chamados com tipo, categoria e prioridade corretos e uma descrição que não exija perguntar tudo de novo.
2. Ver as conversas que precisam de mim, incluindo os casos em que o agent não teve segurança para decidir.
3. Assumir uma conversa, com o agent pausado enquanto eu atendo, e devolvê-la ao agent ou encerrá-la quando terminar.

As histórias 2 e 3 do analista são atendidas pelo console do analista.

O agent deve seguir as regras do service desk (Anexo A) e usar a central de ajuda e os chamados existentes. O sistema de chamados é legado e, de vez em quando, fica indisponível por alguns segundos.

O que construir

O protótipo (Anexo C) mostra as telas e o comportamento esperado. Ele é a referência visual: siga o layout e o estilo, com liberdade nos detalhes.

1. **Login** (Tela 1): e-mail corporativo e senha, aceitando os funcionários dos seus dados de exemplo com qualquer senha.
2. **Chat do funcionário** (Telas 2 e 3): conversa com o Nexo Assist, respostas em tempo real, cards de chamado, seleção de chamado, rascunho para confirmação, perguntas com opções rápidas e limite de 30 mensagens por sessão.
3. **O agent:** decide o que fazer a cada mensagem, usando a central de ajuda e o serviço de chamados, e seguindo as regras do Anexo A.
4. **Serviço de chamados simulado:** com dados de exemplo criados por você (veja o Anexo B), separado do agent, permitindo consultar, criar, atualizar e transferir chamados.
5. **Transferência para analista:** a conversa fica marcada como aguardando atendente, e o agent para de responder até ser liberado.
6. **Console do analista (Telas 4 e 5):** lista de conversas com filtros e status, histórico da conversa, assumir o atendimento (com o agent pausado), devolver ao agent e encerrar, e o painel com os chamados do funcionário.
7. **Visibilidade das decisões:** acima de cada resposta, o protótipo mostra as ações do agent (“consultou seus chamados”, “classificou a prioridade”). Além disso, deve ser possível ver em detalhe o que o agent entendeu, que ação tomou, com que grau de segurança e por quê. O formato desse detalhe é livre (expandir as ações, painel lateral ou log estruturado), desde que seja fácil de ler. Esta é a parte mais importante do desafio.

Requisitos técnicos

Tema	Requisito
Linguagem	TypeScript, com tipagem estrita
Agent	Mastra (preferência) ou LangGraph.js
Modelo	Qualquer provedor de LLM; explique a escolha no README
Interface	Seguir o protótipo do Anexo C (sugestão: Next.js ou React)
Dados	Serviço de chamados simulado, separado do agent
Persistência	Conversas, chamados e o estado de cada conversa (com o agent ou com um analista) devem sobreviver a um reinício da aplicação
Execução	Rodar localmente com poucos comandos, documentados no README, e um .env.example

Também esperamos:

- **Arquitetura clara:** separação entre interface, agent e serviços, com decisões justificadas.
- **Boas práticas:** código organizado, testes onde fazem diferença, tratamento de erros e commits que contem a história do desenvolvimento.
- **Eficiência:** respostas rápidas e uso consciente do modelo, pensando em custo por conversa.
- **Segurança e privacidade:** chaves fora do código e chamados de um funcionário nunca expostos a outro.

Diferenciais

- **Deploy:** publicar a aplicação e enviar o link, para que possamos testar sem rodar o projeto localmente. Pode ser em qualquer plataforma (Vercel, Railway, Fly.io, Render ou outra). Se fizer, proteja as chaves e limite o uso para evitar abuso e custo inesperado, e explique no README como fez o deploy.

Entregáveis e avaliação

O que entregar

1. Link do repositório no GitHub (público ou com acesso liberado para nós).
2. README com: como rodar, visão geral da arquitetura, como o agent decide o que fazer, como você garantiu a qualidade dessas decisões, principais trade-offs e o que faria com mais tempo.
3. Link do deploy, se tiver feito.

Uso de IA: pode e deve usar ferramentas de IA para programar. Na entrevista, vamos conversar sobre o código e pedir pequenas mudanças ao vivo.

Como vamos avaliar

Critério	O que observamos
Qualidade das decisões	O agent entende o que o funcionário quer e escolhe a ação certa, seguindo as regras do service desk
Condução da conversa	Pergunta quando falta informação, confirma antes de agir, mantém o contexto e responde de forma clara
Transparência	É fácil ver e entender o que o agent fez e por quê em cada resposta
Interface	Fidelidade ao protótipo e cuidado com estados de carregamento e erro
Arquitetura e boas práticas	Separação de responsabilidades, tipagem, tratamento de erros, organização e commits
Comunicação	Clareza do README
Diferenciais	Deploy. Conta pontos extras; a ausência não penaliza

Anexo A: regras do service desk

- **Tipos de chamado:** incidente (INC), quando algo parou ou não funciona; requisição (REQ), quando é um pedido de acesso, equipamento ou software.
- **Categorias:** Acesso, Rede e VPN, Hardware, Software, E-mail e Outros.
- **Abertura:** todo chamado precisa de título, descrição, tipo, categoria, prioridade e de quem é afetado. O agent mostra um rascunho e só cria ou atualiza o chamado depois da confirmação do funcionário.
- **Sem duplicidade:** se já existe um chamado aberto sobre o mesmo problema, ele é atualizado, não duplicado. Chamados resolvidos há até 7 dias podem ser reabertos.
- **Transferência:** acontece quando o funcionário pede, em casos críticos, em chamados vencidos ou quando o agent não tem segurança sobre o que fazer. Leva um resumo do caso, e o agent não responde enquanto a conversa está com um analista.
- **Privacidade:** informações de um chamado só podem ser passadas a quem o abriu.
- **Escopo:** apenas assuntos de TI. Outros assuntos (RH, financeiro, facilities) são redirecionados.

Prioridade e prazos. A prioridade combina o impacto (quantas pessoas são afetadas) e a urgência (se há um compromisso em risco). O prazo vale para a primeira resposta e para cada atualização; passado o prazo, o chamado está vencido.

Prioridade	Quando usar	Prazo
Crítica	Algo parado para uma equipe ou para a empresa	1 hora
Alta	Impede uma pessoa de trabalhar, ou há um compromisso no mesmo dia	4 horas úteis
Média	Atrapalha, mas há alternativa	3 dias úteis
Baixa	Pedidos e melhorias sem urgência	5 dias úteis

Anexo B: dados de exemplo

Crie seus próprios dados: alguns funcionários, de 5 a 10 artigos curtos para a central de ajuda e chamados em situações variadas (aberto, vencido, resolvido recentemente, de outros funcionários). Alguns exemplos:

- **Artigo:** “VPN com erro 809: reinstale o cliente de VPN pelo portal de softwares e reinicie o computador. Vale apenas para Windows 10; no Windows 11, abra um chamado.”
- **Artigo:** “Horário do service desk: de segunda a sexta, das 8h às 18h. Chamados críticos têm plantão 24 horas.”

Chamado	Título	Categoria	Prioridade	Status	Última atualização
INC-20311	Notebook lento após atualização do Windows	Hardware	Média	Aguardando analista	Há 6 dias
REQ-10442	Segundo monitor para home office	Hardware	Baixa	Aprovado	Há 1 semana
INC-20150	Senha do SAP bloqueada	Acesso	Média	Resolvido	Há 4 dias

Anexo C: protótipo

Protótipo no Pencil: abrir o protótipo. Os dados que aparecem nele são ilustrativos.

Tela	O que mostra
1. Login	E-mail corporativo e senha, com a apresentação do Nexo Assist
2. Abertura de chamado	Perguntas para completar o chamado, prioridade com prazo e rascunho para confirmação
3. Consulta de chamado	Seleção entre os chamados abertos, status com prazo vencido e oferta de chamar um analista
4. Console: conversa transferida	Lista de conversas, conversa aguardando atendente com o agent pausado e chamados do funcionário
5. Console: analista atendendo	Analista assumiu a conversa e pode devolvê-la ao agent ou encerrar